# CSC336 Assignment 3:Numerical Methods

Hongcheng Huo 1009801626

March 2025

## Question 1

### Error Propagation Formula Derivation

#### Linear System

Consider the linear system:

$$Ax = b. \tag{1}$$

Due to finite precision in computer arithmetic, the computed solution $\tilde{x}$ satisfies:

$$A\tilde{x} = \tilde{b}. \tag{2}$$

We define the errors as:

$$\delta x = \tilde{x} - x, \quad \delta b = \tilde{b} - b. \tag{3}$$

#### Error Propagation Equation

Assuming that $A$ is exact but $b$ contains numerical errors, we substitute $x + \delta x$ and $b + \delta b$ into the equation:

$$A(x + \delta x) = b + \delta b. \tag{4}$$

Rearranging gives:

$$A\delta x = \delta b. \tag{5}$$

Taking norms on both sides:

$$\|\delta x\| = \|A^{-1}\delta b\| \leq \|A^{-1}\|\|\delta b\|. \tag{6}$$

Dividing by $\|x\|$ to normalize the error:

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\| \cdot \frac{\|\delta b\|}{\|b\|} \cdot \frac{\|b\|}{\|x\|}. \tag{7}$$

Since it is common in numerical analysis to approximate:

$$\frac{\|b\|}{\|x\|} \approx \|A\|, \tag{8}$$

we obtain:

$$\frac{\|\delta x\|}{\|x\|} \approx \|A\| \cdot \|A^{-1}\| \cdot \frac{\|\delta b\|}{\|b\|}. \tag{9}$$

#### Condition Number Representation

Using the definition of the **condition number**:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|, \tag{10}$$

we express the error propagation as:

$$\frac{\|\delta x\|}{\|x\|} \approx \kappa(A) \cdot \frac{\|\delta b\|}{\|b\|}. \tag{11}$$

**Machine Precision Influence**

In floating-point arithmetic, numerical errors in $b$ are typically constrained by the machine precision:

$$\frac{\|\delta b\|}{\|b\|} \approx \varepsilon_{\text{machine}}. \tag{12}$$

Substituting this into Equation (11), we obtain:

$$\frac{\|\delta x\|}{\|x\|} \approx \kappa(A) \cdot \varepsilon_{\text{machine}}. \tag{13}$$

**Conclusion**

The final **error propagation formula**:

$$\frac{\|\delta x\|}{\|x\|} \approx \kappa(A) \cdot \varepsilon_{\text{machine}}. \tag{14}$$

This formula indicates:

- Larger condition numbers $\kappa(A)$ amplify errors.
- Floating-point precision $\varepsilon_{\text{machine}}$ affects the accuracy of the computed solution.
- If $\kappa(A) \gg 1$, even a small $\varepsilon_{\text{machine}}$ can lead to significant errors.

Thus, well-conditioned matrices ensure numerical stability, whereas ill-conditioned matrices may require higher precision arithmetic.

# Question 1: Part(a)

We solve the $2 \times 2$ linear system $Ax = b$ using **Gaussian elimination with partial pivoting** on a computer with machine precision $\varepsilon_{\text{machine}} = 10^{-16}$. Given that the condition number of matrix $A$ is:

$$\kappa_\infty(A) \approx 10^6$$

and the exact solution is:

$$x^* = \begin{pmatrix} 12.345678901234567890 \\ 0.001234567890123456 \end{pmatrix}$$

we aim to determine which digits in $x_1$ and $x_2$ are reliable in the computed solution.

## Error Propagation Formula

The relative error in the computed solution is governed by the numerical error propagation formula:

$$\frac{\|\delta x\|}{\|x^*\|} \approx \kappa(A) \cdot \varepsilon_{\text{machine}}$$

where:

- $\|\delta x\|$ represents the error in the computed solution.
- $\|x^*\|$ is the norm of the exact solution.
- $\kappa(A)$ is the condition number of matrix $A$.
- $\varepsilon_{\text{machine}}$ is the machine precision.

## Error Estimation

Substituting the given values:

$$\kappa(A) = 10^6, \quad \varepsilon_{\text{machine}} = 10^{-16}$$

we obtain:

$$\frac{\|\delta x\|}{\|x^*\|} \approx 10^6 \times 10^{-16} = 10^{-10}.$$

This result implies that the relative error in the computed solution is approximately $10^{-10}$, meaning that the computed solution retains at most 10 significant digits.

## Identifying Reliable Digits

Since the relative error is $10^{-10}$, the first **10 digits** of the exact solution can be considered reliable.

### For $x_1 = 12.345678901234567890$

- The first 10 digits are **<u>12.34567890</u>**, which are reliable.
- The remaining digits **1234567890** may be unreliable.

$$x_1 = \underline{12.3456789}01234567890$$

### For $x_2 = 0.001234567890123456$

- The first 10 digits are **<u>0.0012345678</u>**, which are reliable.
- The remaining digits **90123456** may be unreliable.

$$x_2 = 0.00\underline{1234567890}123456$$

## Conclusion

- The error estimate $\frac{\|\delta x\|}{\|x^*\|} \approx 10^{-10}$ suggests that only the first **10 digits** in $x_1$ and $x_2$ are reliable.
- Using the condition number, we determine which digits in the computed solution are likely to match those in the exact solution.
- The final result, with reliable digits underlined, is:

$$x^* = \begin{pmatrix} \underline{12.3456789}01234567890 \\ 0.00\underline{1234567890}123456 \end{pmatrix}$$

# Question 1: (b)

Given a linear system:

$$Mx = b$$

where the components of the exact solution $x$ range from $10^{-1}$ to $10^{3}$ in magnitude, and the condition number of matrix $M$ is:

$$\kappa_\infty(M) \approx 10^8.$$

We seek to determine the required machine precision $\varepsilon_{\text{machine}}$ to ensure that the smallest component of $x$ computed using `scipy.linalg.solve` has a relative error of at most $10^{-6}$.

## Error Propagation Analysis

The relative error in the computed solution is governed by the numerical error propagation formula:

$$\frac{\|\delta x\|}{\|x^*\|} \approx \kappa_\infty(M) \cdot \varepsilon_{\text{machine}}. \tag{15}$$

where:

- $\|\delta x\|$ represents the error in the computed solution.
- $\|x^*\|$ is the norm of the exact solution.
- $\kappa_\infty(M)$ is the condition number of matrix $M$.
- $\varepsilon_{\text{machine}}$ is the machine precision.

To ensure that the smallest component of $x$ has a relative error of at most $10^{-6}$, we impose the condition:

$$\kappa_\infty(M) \cdot \varepsilon_{\text{machine}} \leq 10^{-6}. \tag{16}$$

## Determining Machine Precision

Substituting the given value $\kappa_\infty(M) = 10^8$ into Equation (2), we get:

$$10^8 \cdot \varepsilon_{\text{machine}} \leq 10^{-6}. \tag{17}$$

Solving for $\varepsilon_{\text{machine}}$, we obtain:

$$\varepsilon_{\text{machine}} \leq 10^{-14}. \tag{18}$$

## Interpretation and Conclusion

- The result $\varepsilon_{\text{machine}} \leq 10^{-14}$ implies that single-precision floating point ($\sim 10^{-7}$) is **insufficient**.
- Double-precision floating point ($\sim 10^{-16}$) satisfies the requirement, ensuring that the smallest component of $x$ has a relative error within $10^{-6}$.

Thus, to guarantee the desired accuracy in the computed solution, we conclude that **double-precision floating-point arithmetic (64-bit, IEEE 754) must be used**.

# Question 2: Hilbert Matrix Analysis

## Code Part

```python
import numpy as np
import pandas as pd
from scipy.linalg import hilbert, inv, invhilbert

""" 希尔伯特矩阵（病态矩阵）分析 """
def compute_hilbert_analysis():  1 usage

    # 定义 n 的范围
    n_values = range(2, 13)

    # 计算机的浮点数机器精度
    eps_mach = np.finfo(float).eps / 2

    # 存储计算结果
    results = []

    # 遍历 n 计算相应的值
    for n in n_values:

        # 生成 Hilbert 矩阵
        H_n = hilbert(n)

        # 数值方法求逆矩阵
        H_inv_numeric = inv(H_n)

        # 解析计算的精确逆矩阵
        H_inv_exact = invhilbert(n)

        # 计算相对误差
        relative_error = np.linalg.norm(H_inv_numeric - H_inv_exact,  ord: 2) / np.linalg.norm(H_inv_exact,  ord: 2)
```

```python
    # 遍历 n 计算相应的值
    for n in n_values:

        # 生成 Hilbert 矩阵
        H_n = hilbert(n)

        # 数值方法求逆矩阵
        H_inv_numeric = inv(H_n)

        # 解析计算的精确逆矩阵
        H_inv_exact = invhilbert(n)

        # 计算相对误差
        relative_error = np.linalg.norm(H_inv_numeric - H_inv_exact,  ord: 2) / np.linalg.norm(H_inv_exact,  ord: 2)

        # 计算 Hilbert 矩阵的条件数
        cond_H = np.linalg.cond(H_n,  p: 2)

        # 计算误差与条件数的比值
        error_ratio = relative_error / cond_H

        # 存储结果
        results.append([n, relative_error, cond_H, error_ratio, eps_mach])

    # 转换为 Pandas DataFrame 并打印
    df = pd.DataFrame(results, columns=["n", "Relative Error", "Condition Number", "Error/Cond Ratio", "Machine Epsilon"])
    print(df)

# 运行函数
if __name__ == "__main__":
    compute_hilbert_analysis()
```

## Result Table

```
/usr/local/bin/python3.12 /Users/huohongcheng/Desktop/CSC336 数值方法/CSC336 数值方法 Assignment 3/Question2.py
     n  Relative Error  Condition Number  Error/Cond Ratio  Machine Epsilon
0    2    2.919507e-16      1.928147e+01      1.514152e-17     1.110223e-16
1    3    3.742681e-15      5.240568e+02      7.141748e-18     1.110223e-16
2    4    9.841574e-14      1.551374e+04      6.343780e-18     1.110223e-16
3    5    7.978114e-13      4.766073e+05      1.673939e-18     1.110223e-16
4    6    3.973049e-11      1.495106e+07      2.657370e-18     1.110223e-16
5    7    7.688192e-10      4.753674e+08      1.617316e-18     1.110223e-16
6    8    5.863012e-08      1.525758e+10      3.842689e-18     1.110223e-16
7    9    4.218510e-06      4.931535e+11      8.554152e-18     1.110223e-16
8   10    1.268960e-04      1.602503e+13      7.918614e-18     1.110223e-16
9   11    2.591193e-03      5.216249e+14      4.967540e-18     1.110223e-16
10  12    4.746503e-02      1.714697e+16      2.768129e-18     1.110223e-16

Process finished with exit code 0
```

## Analysis

The function `scipy.linalg.inv(`$H_n$`)` produces an inaccurate approximation to $H_n^{-1}$ due to the **ill-conditioned of the Hilbert matrix**.

### Hilbert Matrices Have Extremely High Condition Numbers

The **condition number** of $H_n$ grows exponentially as $n$ increases, making numerical computations highly unstable. From the computed results:

- $\text{cond}(H_3) \approx 5.24 \times 10^2$ (moderate instability)
- $\text{cond}(H_{10}) \approx 1.60 \times 10^{13}$ (extreme instability)
- $\text{cond}(H_{12}) \approx 1.71 \times 10^{16}$ (near the limit of double-precision floating-point arithmetic)

### Floating-Point Precision Limitations

Python uses **IEEE 754 double-precision floating-point numbers**, where the smallest distinguishable difference (machine epsilon) is approximately $10^{-16}$. For $n \geq 10$, the condition number exceeds $10^{13}$, meaning small numerical errors **get amplified beyond machine precision**, leading to visible inaccuracies.

### Instability of Numerical Inversion Methods

`scipy.linalg.inv(`$H_n$`)` relies on **Gaussian elimination (LU decomposition)**, which is susceptible to round-off errors. These errors get **magnified by the high condition number**, causing large deviations in the computed inverse. From the computed table, the **relative error grows rapidly**:

- $n = 6$, relative error: $3.97 \times 10^{-11}$
- $n = 10$, relative error: $1.27 \times 10^{-4}$
- $n = 12$, relative error: $4.75 \times 10^{-2}$ (**significant error**)

### Conclusion

The **high condition number of Hilbert matrices amplifies numerical errors**, making `scipy.linalg.inv(`$H_n$`)` unreliable for large $n$. The **relative error increases exponentially**, reaching nearly **5% at** $n = 12$.

# Question 3: Pascal matrix Analysis

## Code Part

```python
import numpy as np
import pandas as pd
import scipy.linalg


def generate_system(n):  1 usage
    """ Generate the Pascal matrix A, the exact solution vector x*, and the right-hand side vector b. """

    # Generate Pascal matrix A (生成 Pascal 矩阵 A)
    A = scipy.linalg.pascal(n)

    # Generate exact solution vector x* (random values) (生成随机精确解向量 x* (随机数))
    x_star = np.random.rand(n, 1)

    # Compute the right-hand side vector (计算右端项向量 b = A * x_star)
    b = A @ x_star

    return A, x_star, b

```

```python
def solve_and_evaluate(n):  1 usage
    """ Solve Ax = b and compute errors, condition number, and determinant. """
    A, x_star, b = generate_system(n)

    # 使用 SciPy 求解 Ax = b
    x = scipy.linalg.solve(A, b)

    # 计算相对误差 ||x - x*|| / ||x*||
    # Compute relative error
    relative_error = np.linalg.norm(x - x_star) / np.linalg.norm(x_star)

    # 计算残差 r = b - Ax & 相对残差 ||r|| / ||b||
    # Compute residual and relative residual
    residual = b - A @ x
    relative_residual = np.linalg.norm(residual) / np.linalg.norm(b)

    # 计算矩阵的条件数和行列式
    # Compute condition number and determinant
    cond_A = np.linalg.cond(A)
    det_A = np.linalg.det(A)

    return n, relative_error, relative_residual, cond_A, det_A
```

```python
def run_experiment():  1 usage
    """
    Run the experiment, gradually increasing n, computing and printing errors,
    residuals, condition number, and determinant until the relative error exceeds 1.
    """
    results = []  # Store results for different values of n
    n = 1         # Start with n=1

    while True:
        n, rel_err, rel_res, cond_A, det_A = solve_and_evaluate(n)
        results.append([n, rel_err, rel_res, cond_A, det_A])

        # Stop experiment if relative error exceeds 1
        if rel_err > 1:
            break

        n += 1  # Increase n and continue

    # Convert results to a Pandas DataFrame and display as a table
    df = pd.DataFrame(results, columns=["n", "Relative Error", "Relative Residual", "Condition Number cond(A)",
                                        "Determinant det(A)"])

    print("\n Experiment Results:\n")
    print(df.to_string(index=False))

    return df

if __name__ == "__main__":
    run_experiment()
```

## Result Table

```
Experiment Results:

n  Relative Error  Relative Residual  Condition Number cond(A)  Determinant det(A)
1    0.000000e+00       0.000000e+00              1.000000e+00            1.000000
2    0.000000e+00       0.000000e+00              6.854102e+00            1.000000
3    1.465634e-15       0.000000e+00              6.198387e+01            1.000000
4    1.671620e-14       1.045304e-16              6.919374e+02            1.000000
5    3.782129e-14       2.233229e-16              8.517524e+03            1.000000
6    3.397339e-13       2.132869e-16              1.107867e+05            1.000000
7    2.030262e-12       6.954456e-17              1.493442e+06            1.000000
8    3.567254e-11       1.288617e-16              2.064517e+07            1.000000
9    2.291910e-10       2.447818e-16              2.907827e+08            1.000000
10   1.429653e-08       1.844586e-17              4.155206e+09            1.000000
11   2.750749e-09       1.684420e-16              6.006353e+10            1.000000
12   1.439525e-07       1.552375e-16              8.763949e+11            1.000000
13   3.140403e-06       5.757502e-18              1.288757e+13            1.000001
14   9.932076e-05       1.430645e-16              1.907634e+14            0.999968
15   1.326823e-03       1.555951e-16              2.839449e+15            1.000538
16   5.647851e-03       4.754407e-18              4.248931e+16            1.002779
17   3.195577e-01       1.434988e-16              6.377293e+17            1.004151
18   2.740436e-02       1.503376e-17              9.299854e+18            0.712616
19   1.006031e+01       1.735130e-16              1.366271e+20            2.952457

Process finished with exit code 0
```

## Analysis

**Determinant** $\det(M)$: Indicates whether a matrix is invertible but does not quantify numerical stability.

**Condition Number cond**$(M)$:

$$\text{cond}(M) = \|M\| \cdot \|M^{-1}\| \tag{19}$$

measures how sensitive the solution $x$ of $Ax = b$ is to small changes in $A$ or $b$:

$$\frac{\|\Delta x\|}{\|x\|} \approx \text{cond}(M) \cdot \frac{\|\Delta b\|}{\|b\|} \tag{20}$$

A large $\text{cond}(M) \gg 1$ indicates that numerical errors are greatly amplified, making the matrix ill-conditioned.

### Determinant Not a Reliable Indicator

- **A small determinant does not necessarily indicate an ill-conditioned matrix**:
  - Consider a diagonal matrix $M = 10^{-10}I$, whose determinant is:

    $$\det(M) = (10^{-10})^n = 10^{-10n} \tag{21}$$

    Although this is very small, the condition number is:

    $$\text{cond}(M) = 1 \tag{22}$$

    indicating that the matrix is well-conditioned despite having a small determinant.

- **A determinant close to 1 does not imply a well-conditioned matrix**:
  - Our experiment on Pascal matrices demonstrates that even when $\det(A) \approx 1$, the condition number $\text{cond}(A)$ can be extremely large (on the order of $10^{15}$), leading to highly unstable solutions.

**Experimental Data**

| $n$ | $\det(A)$ | $\text{cond}(A)$ | Relative Error |
|---|---|---|---|
| 1 | 1.00 | 1.00 | 0.00 |
| 5 | 1.00 | $8.52 \times 10^3$ | $4.27 \times 10^{-14}$ |
| 10 | 1.00 | $4.15 \times 10^9$ | $2.05 \times 10^{-8}$ |
| 15 | 1.00 | $2.83 \times 10^{15}$ | $1.10 \times 10^{-3}$ |
| 18 | 0.71 | $9.30 \times 10^{18}$ | $7.02 \times 10^{-2}$ |
| 19 | 2.95 | $1.37 \times 10^{20}$ | $3.49 \times 10^1$ |

**Observations**:

- When $\det(A) \approx 1$, the condition number $\text{cond}(A)$ can still be extremely large ($10^9$ or more), indicating severe ill-conditioning.
- The determinant does not change significantly, yet errors explode, proving that $|\det(M)|$ is not a reliable indicator of conditioning.

**Conclusion**

- $|\det(M)|$ **is not a reliable measure of matrix conditioning**; the true measure is the condition number $\text{cond}(M)$.
- A small determinant does not necessarily indicate an ill-conditioned matrix, nor does a large determinant imply a well-conditioned matrix.
- In numerical computations, the condition number $\text{cond}(M)$ should be used to assess matrix stability rather than the determinant.

# Question 4: Polynomial Interpolation

## Polynomial Interpolation Using the Monomial Basis

We assume the interpolating polynomial is a cubic polynomial:

$$P(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$$

Given the data points:

| $x$ | 1 | 2 | 3 | 6 |
|---|---|---|---|---|
| $y$ | $-2$ | $-1$ | $-22$ | $-37$ |

Substituting these points into the polynomial equation:

$$\begin{aligned}
P(1) &= a_0 + a_1(1) + a_2(1^2) + a_3(1^3) = -2 \\
P(2) &= a_0 + a_1(2) + a_2(2^2) + a_3(2^3) = -1 \\
P(3) &= a_0 + a_1(3) + a_2(3^2) + a_3(3^3) = -22 \\
P(6) &= a_0 + a_1(6) + a_2(6^2) + a_3(6^3) = -37
\end{aligned}$$

This gives the linear system:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 27 \\ 1 & 6 & 36 & 216 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \\ -22 \\ -37 \end{bmatrix}$$

Solving this system, we obtain:

$$a_0 = -43, \quad a_1 = 67, \quad a_2 = -29, \quad a_3 = 3$$

Thus, the interpolating polynomial is:

$$P(x) = 3x^3 - 29x^2 + 67x - 43$$

## Lagrange Interpolation

The Lagrange interpolating polynomial is given by:

$$P(x) = \sum_{i=0}^{n} y_i L_i(x)$$

where the Lagrange basis functions are:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Computing each $L_i(x)$:

$$L_0(x) = \frac{(x-2)(x-3)(x-6)}{(1-2)(1-3)(1-6)} = \frac{(x-2)(x-3)(x-6)}{10}$$

$$L_1(x) = \frac{(x-1)(x-3)(x-6)}{(2-1)(2-3)(2-6)} = \frac{(x-1)(x-3)(x-6)}{4}$$

$$L_2(x) = \frac{(x-1)(x-2)(x-6)}{(3-1)(3-2)(3-6)} = \frac{(x-1)(x-2)(x-6)}{-6}$$

$$L_3(x) = \frac{(x-1)(x-2)(x-3)}{(6-1)(6-2)(6-3)} = \frac{(x-1)(x-2)(x-3)}{60}$$

Substituting into the Lagrange formula:

$$P(x) = -2L_0(x) - 1L_1(x) - 22L_2(x) - 37L_3(x)$$

After simplification, we obtain:

$$P(x) = 3x^3 - 29x^2 + 67x - 43$$

which is identical to the result from part (a).

## Newton Interpolation

The Newton interpolating polynomial is given by:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2)$$

where the coefficients $a_i$ are computed using the divided difference table:

| $x$ | $y$ | $f[x_i, x_{i+1}]$ | $f[x_i, x_{i+1}, x_{i+2}]$ | $f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$ |
|---|---|---|---|---|
| 1 | $-2$ | | | |
| 2 | $-1$ | $\frac{-1-(-2)}{2-1} = 1$ | | |
| 3 | $-22$ | $\frac{-22-(-1)}{3-2} = -21$ | $\frac{-21-1}{3-1} = -11$ | |
| 6 | $-37$ | $\frac{-37-(-22)}{6-3} = -5$ | $\frac{-5-(-21)}{6-2} = 4$ | $\frac{4-(-11)}{6-1} = 3$ |

Thus, the Newton interpolating polynomial is:

$$P(x) = -2 + 1(x - 1) - 11(x - 1)(x - 2) + 3(x - 1)(x - 2)(x - 3)$$

Expanding:

$$P(x) = 3x^3 - 29x^2 + 67x - 43$$

which matches the results from (a) and (b).

## Conclusion

Using three different interpolation methods, we obtained the same polynomial:

$$P(x) = 3x^3 - 29x^2 + 67x - 43$$

This confirms the equivalence of the monomial basis, Lagrange, and Newton interpolation methods.